

无服务器计算的研究综述

谢昌熔

国防科技大学计算机学院，23020119

摘 要：无服务器计算是一种新兴的云计算范式，被广泛应用于开发各类应用程序，它允许开发者在函数的粒度上只关注应用程序逻辑代码，从而将开发人员从基础设施管理中解放出来。与传统的单体应用架构以及微服务架构相比，无服务器计算有其特殊之处，这为它带来了特殊的优势与挑战。本文对无服务器计算的研究现状进行了综述，通过总结近年来无服务器计算的相关研究，分析了它作为一种新兴架构模式能快速流行的原因，评述了无服务器计算的研究现状，最后总结了当前亟待探索的方向。本文的研究成果为研究人员全面掌握无服务器计算的研究现状，探究更有价值的方向提供了指导。

关键词：无服务器计算，文献综述，云计算

1 引言

无服务器计算是一种新兴的云计算范式，允许开发者在云计算平台中开发、部署和运行应用程序，而无需分配和管理服务器等资源或考虑运维方面操作[1]。当前无服务器计算已经被广泛应用于各种领域，例如机器学习[2]、数值计算[3]、视频处理[4]、物联网[5]、大数据分析[6]等。无服务器计算使开发者只需要专注于软件的应用逻辑，而将纷繁复杂的底层资源分配管理和环境配置等工作交给云计算平台[7]。除此之外，无服务器计算按实际使用的资源收费，避免了传统的包月包年租借模式下资源空闲时造成不必要开销。随着无服务器计算的广泛流行，许多云计算服务供应商都推出了自己的无服务器计算平台，例如 AWS Lambda[8]、Microsoft Azure Functions[9]、Google Cloud Functions[10]。此外还有一些开源无服务器计算平台，如 OpenWhisk[11]、OpenFaaS[12]。

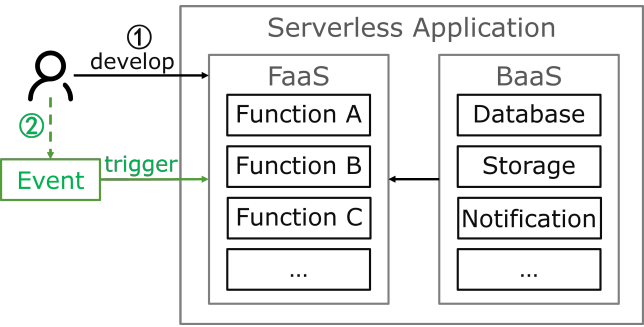


图 1 无服务器计算组成结构

无服务器计算本质上可以视作函数即服务

(FaaS) 与后端即服务 (BaaS) 的结合 [13]，如图 1 所示。函数即服务指的是云计算平台提供的函数服务。开发者需要在函数的粒度上开发应用，并为函数设置事件触发器，当指定事件发生时云计算平台加载函数运行，运行结束后将函数运行的容器销毁。[14]。后端即服务指的是云计算平台提供的一系列后端服务，例如对象数据库、身份鉴权功能等。通过将常用功能集成并提供，减少开发者的重复工作量，加快软件开发流程。相比以往的云计算范式，无服务器计算具有事件驱动、无状态、自动放缩、资源使用限制等特点 [15]。同时这也导致诸多挑战的存在，例如冷启动延迟、对通信需求高、难以调试与测试等问题 [16]。

由于无服务器计算的光明前景，越来越多的研究开始关注这一新范式，使无服务器计算相关研究在近年来取得了长足进步，包括资源管理 [17]、性能优化 [18]、编程语言框架 [19] 等领域。本文旨在综述无服务器计算研究的现状与趋势，以帮助研究人员更好地掌握这一重要研究领域的相关工作。本文剩余章节组织如下。第二节介绍本文的研究方法，第三节讨论无服务器计算的特点，第四节分析无服务器计算主要研究方向与成果，第五节总结当前研究的局限性并展望未来方向。

2 研究方法

本文主要研究以下三个问题。

- 无服务器计算与传统应用架构相比有什么特

点？

该研究问题关注无服务器计算与已有的单体应用架构与微服务架构等的特殊之处以及带来的挑战。对这一问题的研究有利于理解无服务器计算的本质以及解决这种新范式带来的特殊挑战。

• 无服务器计算的研究现状如何？

该研究问题关注无服务器计算的现有研究以及成果。对这一问题的研究有利于全面系统地掌握当前无服务器计算的主流研究领域，亟待解决的问题，帮助研究人员更快地找到研究重点领域。

• 无服务器计算当前的研究有何不足之处，未来可能的研究方向有哪些？

该研究问题在总结了无服务器研究现状的基础上，分析现有研究的局限性，并展望未来的工作。对这一问题的研究有助于填补当前研究的不足，并为研究人员未来的方向提供指导。

本文通过在 Google Scholar、DBPL、ACM Digital library、IEEE Explore 等学术文献资料库与检索工具中，检索关键词“serverless”、“FaaS”、Function-as-a-service “来获取文献资料。并通过阅读其标题与摘要，基于下列原则对其进行筛选：（1）分析本文是否与无服务器计算相关，剔除不相关的文献（2）关注文章的来源，只留下收录于水平较高的期刊与会议的论文（3）只关注近五年内的论文，剔除 2019 年之前的相关论文，优先选取近三年内的文献资料。

最终筛选得到了本文研究的文献资料库，涵盖多个领域、多种来源，基于这些文献探究上述的研究问题。

3 无服务器计算的特点

无服务器计算并非无需服务器运行的应用，其本质上是由云服务供应商承担复杂易错的资源管理与运维责任，从而减少应用开发者的工作，降低应用开发成本，在开发者的视角来看不必考虑服务器运维管理相关的工作。作为一种新兴的云计算范式，无服务器计算与传统的单体应用结构与微服务结构相比，具有由事件驱动、专注于函数、计算存储分离、自动放缩、资源使用限制等特点。

3.1 无服务器计算特点

3.1.1 事件驱动

如图 1 所示，无服务器计算模式下的应用由多个函数组成，这些函数由指定事件触发，例如 http 请求，访问 api 等都可作为触发事件。无服务器计算函数的调用周期如图 2 所示，控制器收到指定事件发生信号后，无服务器计算平台将为函数分配容器，如 Docker 等，并从代码数据库中加载函数代码以及函数运行环境，随后执行函数，待函数执行完毕后，无服务器计算平台将函数执行的容器销毁并回收以供新的函数请求使用。[20]。

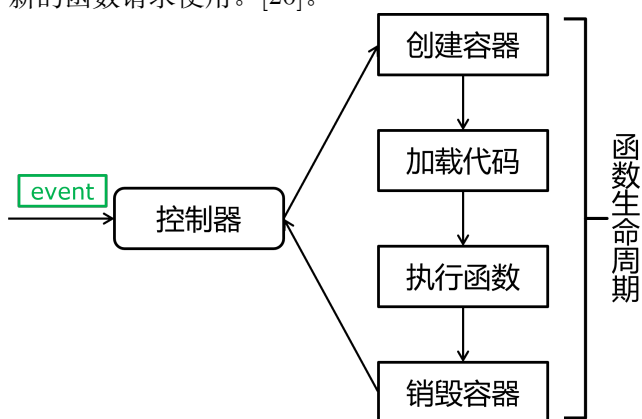


图 2 无服务器计算函数调用周期

传统的单体应用和微服务应用都是持续运行，无服务器计算应用则是间断运行，由事件触发。这一特点导致其相比传统应用能更好地节省资源，避免应用空闲时造成的资源浪费，并且适用于按使用资源的付费模式，降低了云上应用的成本。

3.1.2 专注于函数

在无服务器计算下，开发者可以自由选取自己熟悉的语言来编写函数代码，而无需考虑开发环境配置等耗费时间的问题。利用无服务器计算平台提供的 IDE 和部署机制，开发者可以轻易实现应用从开发到部署全过程 [21]。由于开发者只需要专注于函数的功能逻辑代码编写，相比传统应用开发模式，避免了复杂的开发环境配置与部署过程，提高开发效率。同时由于应用部署在无服务器计算平台上，可以借助平台提供的大量运维工具，从而减少运维成本。

无服务器计算平台除可以为开发者提供各种开发环境以外，还可以提供大量后端服务，例如数据存储、负载均衡、访问鉴权等。这些服务可以由开发人员在应用程序中直接通过 api 使用，避免开发人员重复开发类似功能导致的成本浪费。

无服务器计算平台	AWS Lambda	Microsoft Azure Functions	Google Cloud Functions	Openwhisk
可使用内存范围	128MB-10240MB	1536MB	128,256,512,1024,2048,4096MB	128MB-2048MB
代码包大小	250MB(未压缩) 50MB(压缩)	无限制	500MB(未压缩) 100MB(压缩)	48MB(未压缩)
函数运行时间上限	900 秒	600 秒	540 秒	600 秒

表 1: 常见无服务器计算平台资源使用限制

3.1.3 计算存储分离

由于无服务器计算平台会在函数结束后将其容器销毁, 因此无服务器计算应用不具备暂时存储数据的能力, 只能借助独立于计算函数的外部存储库。这种计算与存储分离的结构导致无服务器计算应用是一种无状态应用, 即不能临时保存应用状态。

计算与存储分离的方式可以确保无服务器计算平台对突发性工作负载的自动扩展能力。与传统计算与存储在同一容器的应用相比, 扩展其计算能力更为容易。然而, 这一特点也导致无服务器计算应用更依赖于外部提供的存储服务, 限制了无服务器应用在部分场景的使用, 例如迭代训练的机器学习模型中通常需要保存父代的模型再开始训练子代, 因此不适合采用无服务器计算。

3.1.4 自动放缩

无服务器计算应用可以根据当前的工作负载需求动态调整计算资源分配。调整方式分为垂直放缩与水平放缩两种 [22]。垂直放缩是通过调整单个计算单元的计算能力来放缩, 例如为某一单元分配更多的计算资源等。水平放缩则是通过增加计算单元来放缩, 例如为某一任务分配更多的计算单元。在实际应用中, 由于调整分配计算单元数量更为容易, 云服务供应商通常使用水平放缩的方式。

无服务器计算自动放缩的特性是其得到广泛流行的主要原因。这一特点使其在流量高峰时可以应对大量突发的工作负载, 在流量低谷时又可以避免空闲资源浪费。合理的自动放缩策略需要预先计算即将出现的工作负载, 否则可能会由于分配计算单元造成不必要的时间开销, 这一问题仍未得到很好地解决 [23]。

3.1.5 资源使用限制

无服务器计算是一种多租户模式, 其平台上同时运行大量的应用与函数, 为防止某一函数占用过多资源, 保持无服务器计算自动扩展能力, 无服务器计算平台通常会为每个函数设置资源使用限制, 例如可使用内存范围、代码包大小、函数运行时间上限等。表 1 列出了四个常见无服务器计算平台 AWS Lambda、Microsoft Azure Functions、Google Cloud Functions、Openwhisk 的资源使用限制, 由于无服务器计算函数执行时间大多数为十几秒 [24], 因此这些平台设置的上限都为百秒。

传统的应用资源使用只受到硬件条件的限制, 而无服务器计算应用则需要受到平台的约束。这导致无服务器计算在许多应用领域受到限制。例如由于代码包大小上限的限制, 将无服务器计算应用于大型项目时, 需要进行设计将其拆分为多个组件。此外, 这一特性也导致无服务器计算的调试与测试更加困难。##

4 无服务器计算研究现状

本节的工作是基于筛选得到的论文, 探究无服务器计算研究的现状与趋势。通过综述分析近年来的无服务器计算研究, 总结热门研究领域与方向, 为研究者全面系统地掌握无服务器计算研究提供帮助。

4.1 无服务器计算研究方向概述

相比传统的架构模式, 无服务器计算广泛流行的原因主要在于其商业价值, 可以降低软件的开发成本, 加速开发周期, 因此在无服务器计算的研究中, 性能、开销方面的问题持续受到关注。除此之外, 由于它将应用划分为函数粒度的单元, 因此函数调度策略对无服务器计算也十分关键, 适宜的函数调度策略有利于提高资源利用率, 减少因分配函数运行容器、加载函数代码所耗费的时间。

根据无服务器计算当前研究的领域,可分为如下几个主要研究方向。表 2 总结了这些研究方向的概况。

- **无服务器计算资源管理**

资源管理的任务是为无服务器计算函数分配适当的资源,并为无服务器计算函数调度合适的运行容器,这一过程与无服务器计算应用的性能、成本、安全稳定性密切相关 [25]。与之对应,当前对这一领域的关注也主要分为资源分配策略与函数调度策略两部分。

- **无服务器计算性能**

无服务器计算的性能研究分为性能优化与性能预测。当前困扰无服务器计算的一个重要问题是分配函数运行容器与加载代码所耗费的时间成本较高,给无服务器计算增加了不必要的成本,性能优化主要解决这方面的问题。由于无服务器计算平台或应用程序的性能差异会影响用户的选择,因此一些研究也针对平台或应用程序进行性能预测 [7]。

- **无服务器计算费用**

与传统云平台应用相关,无服务器计算采用独特的计费模式,即按资源使用量付费。费用与无服务器计算函数的执行时间、分配或使用的内存大小以及调用次数密切相关。研究人员对如何基于这些因素和其他可能的影响因素来预测和优化无服务器计算或程序的费用进行了探索##。

- **无服务器计算应用开发**

无服务器计算作为一种新的云计算范式,应用开发方式与以往的服务器模式有较大的不同,例如其只需要考虑功能逻辑代码,而无需考虑资源管理等。为了降低开发者从传统模式转向无服务器计算的学习成本,同时促进无服务器计算的普及,研究人员提出了许多无服务器计算编程框架,以适应不同的需求。除此之外,将传统应用重构为无服务器计算架构也是这一方面研究的主要问题之一 [26]。

4.2 无服务器计算资源管理

4.2.1 资源分配策略

在无服务器计算模式下,内存、CPU 等资源都由无服务器平台进行分配。高效灵活的资源管理可以应付资源和延迟要求不同的各种无服务器应用程序,同时提高资源利用率,最大限度降低云计算供应商的成本。

这些方法大多数是通过预估无服务器应用程序的资源消耗量,以分配适宜的资源量。例如通过分析历史运行的数据,探究如何在无服务器计算平台中动态调整 CPU 等资源。Kim 等人提出了一种细粒度的 CPU 上限控制器,可动态调整 CPU 使用限制,以最大限度减少资源争夺 [27]。

4.2.2 函数调度策略

函数调度策略指的是对无服务器计算函数机械能分析,根据估计的资源消耗和运行寿命,对无服务器计算函数进行调度,从而设计出最节省资源的无服务器计算函数工作流。除此之外,此类策略也可以针对无服务器计算函数的特性,为其分配合适的运行容器,例如为功能为深度学习的函数分配底层处理器为 GPU 的容器。

最近的研究提出了许多函数级别的调度器, [28], [29], 它们通过对无服务器计算函数进行分类,以便于调度,同时还可以调整无服务器计算函数容器的共享资源,使资源调整更为灵活。

4.3 无服务器计算性能

4.3.1 性能优化

无服务器计算性能优化主要是减少启动时延迟,即冷启动延迟。冷启动延迟指的是当事件发生触发无服务器计算函数时,无服务器计算平台从而开始准备环境的过程产生的延迟。这一过程包括分配容器、拉取环境、加载代码等。研究显示,这一过程造成了无服务器计算函数较大的时间开销浪费 [41]。

当前缓解冷启动延迟主要方法是维护“热”容器和设计架构降低冷启动延迟。维护“热”容器指的是在无服务器计算函数执行结束后不销毁容器,而是将环境保存一段时间,以供下次使用 [30]。然而这种方式的局限性是难以选择被维护与被销毁的容器。设计架构方式则是通过设计策略缓解冷启动问题。例如 WLEC[31] 采用类缓存的方法保存被认为最经常

研究方向	研究内容	研究特点	代表性工作
无服务器计算 资源管理	资源分配策略	根据无服务器计算应用程序的性能要求相似性等指标设计动态资源分配策略。	一种自动细粒度 CPU 分配策略 [27]
	函数调度策略	通过分析无服务器计算函数的资源消耗和寿命, 依据此对无服务器函数进行调度。	Fnsched[28], Ensure[29]
无服务器计算 性能	性能优化	主要利用局部性原理设计函数调度顺序或维护“热”容器来降低启动延迟	“热”容器池研究 [30], 架构设计降低冷启动 [31]
	性能预测	通常采用的预测方法是基于回归模型或基于统计学习	基于回归模型的方法 [32], 基于统计学习的方法 [33]
无服务器计算 费用	费用预测	由于无服务器计算按使用资源付费的特性, 某些开发者需要预先估计无服务器计算应用程序可能使用的费用, 因此对计算费用进行预测	基于回归模型的预测 [34], 基于统计学习的预测 [35]
	费用优化	实现费用优化主要是通过设计分配资源更适宜的分配方法来实现	无服务器计算与虚拟机结合的方法 [36], 动态资源分配 [37]
无服务器计算 应用开发	编程框架	编程框架研究分为通用编程框架与特定领域编程框架两类, 前者可以加快开发周期, 后者使无服务器计算适用于更多领域	通用框架 [38], 特定框架 [5]
	应用重构	通过设计自动或手动重构方法, 将传统软件重构为无服务器模式, 为重构软件提供指导或自动化工具以减轻开发者负担	手动重构方法 [39], 自动重构方法 [40]

表 2: 无服务器计算研究现状概况

调用的容器。

4.3.2 性能预测

无服务器计算的性能预测研究主要目的是为设计更优秀的调度策略等提供支持。利用无服务器计算应用程序的历史数据预测其性能。然而在实际应用中,由于应用程序的历史数据较难获得,因此这种方法仍然有一定的局限性。

预测方法分为基于回归模型的方法与基于统计学习的方法两种。基于回归模型的方法需要利用历史数据训练模型,例如利用 CPU 运行时间预测其性能 [32]。基于统计学习的预测方法主要是预测无服务器计算平台的性能,利用一些关键参数,如冷启动率、冷启动延迟等来预测无服务器函数或应用程序的响应时间 [33]。

4.4 无服务器计算费用

4.4.1 费用预测

无服务器计算费用预测主要针对一些计划性较强的商业公司。这些公司通常需要预先估计所使用的云计算服务的费用,然而无服务器计算按使用资源付费的特点决定其只能通过预测的方式计算可能产生的费用。

当前无服务器计算费用预测的方法与性能预测的方法相似,分为基于回归模型和统计学习两种。例如利用对无服务器应用程序运行的监测数据,采用回归模型进行预测 [34]。除此之外,COSE 框架 [35] 基于统计学习,跟踪无服务器函数的费用和一些隐藏配置的关系,以此预测不同配置下的成本。

4.4.2 费用优化

无服务器计算应用程序通常由多个无服务器计算函数组成,每次调用这些函数都会产生相应的费用,如何设计策略帮助开发者减少费用是该领域的热门方向。

一种优化策略是将无服务器计算与虚拟机结合,在保证性能约束的同时选取两种模式中的最佳方案 [36]。此外,Spillner 测量了无服务器计算函数在特定时间内的内存消耗情况,以为函数分配最合适的内存,避免资源浪费 [37]。

4.5 无服务器计算应用开发

4.5.1 编程框架

无服务器计算的局限性促使研究人员提出新的编程框架来服务于某些特定应用领域。除此之外,作为一种新兴的范式,通用无服务器计算编程框架的提出,也有利于降低开发者的学习使用成本,加快其开发速度,同时这样一种高层的抽象框架也可以有效应对无服务器计算的“供应商锁定”问题。

无服务器计算的编程框架分为通用框架与特定领域框架。通用框架旨在解决一些无服务器计算的通用问题或提高开发效率,例如 Wukong[38] 框架是一个面向无服务器计算的去中心化框架,采用了融合静态调度与动态调度的去中心调度机制。以此提高应用程序性能和资源利用率。特定领域框架则是针对一些特定应用场景的编程框架。例如 Jindal 等人引入了 FaaS 的扩展框架,通过功能交付网络解决异构集群的数据访问问题 [5]。

4.5.2 应用重构

无服务器计算是一种全新的架构模式,在这种模式下,应用的开发与代码结构都与以往的架构模式不同,因此将原有的应用迁移到无服务器计算模式下是一件有挑战性的工作。这项任务需要同时具备传统应用与无服务器计算两个领域的丰富知识,因此重构为无服务器计算应用的指导方法获得了广泛关注。

无服务器计算应用程序的重构方式分为手动重构和自动重构两类。手动重构旨在为开发者将应用程序重构为无服务器计算架构提供方法指导。Christidis 等人通过考虑一些限制因素来处理人工智能相关的应用程序,提供了一种手动重构方法 [39]。自动重构则是利用自动化方法对应用程序进行重构,例如 SEAPORT 是一种自动评估方法,基于规范的无服务器应用程序模型对软件进行转换 [40],然而当前自动化方法的效果仍有待提升。

5 无服务器计算研究总结与展望

本文主要对无服务器计算的研究现状进行了文献综述,主要关注了无服务器计算的特点与研究现状以及成果。在这些研究综述的基础上,无服务器计算仍然有以下几个方向可供研究,以解决当前无服务器计算亟待解决的问题。

• 冷启动延迟问题

尽管当前的冷启动优化方案可以有效降低冷启动延迟，但它们主要针对容器创建或运行时启动的加速，此类方法容易导致容器缺乏隔离性和灵活性。此外，应用程序初始化时加载代码耗费的时间也是一个不可忽视的问题。

- **通用的应用程序转换方法**当前无服务器计算的流行的较大阻碍是，许多广泛使用的应用程序都采用传统的架构模式，将它们转化为无服务器计算架构是一项艰巨的工作。因此一种通用的无服务器计算应用程序转换方法具有十分重要的意义。
- **通用的无服务器计算应用程序数据集**尽管已经有大量的无服务器计算相关研究出现，但该领域仍然缺乏一个通用的无服务器应用程序数据集 [15]。因此，一个囊括了广泛的无服务器计算应用程序类型的基准数据集对无服务器计算领域研究至关重要。

参考文献

- [1] S. Kounev, N. Herbst, C. L. Abad, *et al.*, “Serverless computing: What it is, and what it is not?” *Communications of the ACM*, vol. 66, no. 9, pp. 80–92, Sep. 2023.
- [2] H. Wang, D. Niu, and B. Li, “Distributed machine learning with a serverless architecture,” in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, 2019, pp. 1288–1296.
- [3] V. Shankar, K. Krauth, K. Vodrahalli, *et al.*, “Serverless linear algebra,” in *Proceedings of the 11th ACM Symposium on Cloud Computing*, Association for Computing Machinery, 2020, pp. 281–295, ISBN: 9781450381376. [Online]. Available: <https://doi.org/10.1145/3419111.3421287>.
- [4] L. Ao, L. Izhikevich, G. M. Voelker, and G. Porter, “Sprocket: A serverless video processing framework,” *Proceedings of the ACM Symposium on Cloud Computing*, 2018.
- [5] A. Jindal, M. Gerndt, M. Chadha, V. Podolskiy, and P. Chen, “Function delivery network: Extending serverless computing for heterogeneous platforms,” *Software: Practice and Experience*, vol. 51, no. 9, pp. 1936–1963, 2021.
- [6] J. Enes, R. R. Expósito, and J. Touriño, “Real-time resource scaling platform for big data workloads on serverless environments,” *Future Generation Computer Systems*, vol. 105, pp. 361–379, 2020.
- [7] J. Wen, Z. Chen, X. Jin, and X. Liu, *Rise of the planet of serverless computing: A systematic review*, Dec. 2022. arXiv: 2206.12275 [cs].
- [8] AWS Lambda. <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>.
- [9] Microsoft Azure Functions. <https://docs.microsoft.com/en-us/azure/azure-functions/>.
- [10] Google Cloud Functions. <https://cloud.google.com/functions>.
- [11] OpenWhisk. <https://openwhisk.apache.org/>.
- [12] OpenFaaS. <https://www.openfaas.com/>.
- [13] E. Jonas, J. Schleier-Smith, V. Sreekanti, *et al.*, *Cloud programming simplified: A berkeley view on serverless computing*, Feb. 2019. arXiv: 1902.03383 [cs].
- [14] P. Leitner, E. Wittern, J. Spillner, and W. Hummer, “A mixed-method empirical study of function-as-a-service software development in industrial practice,” *Journal of Systems and Software*, vol. 149, pp. 340–359, Mar. 2019.
- [15] J. Wen, Z. Chen, Y. Liu, *et al.*, “An empirical study on challenges of application development in serverless computing,” in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, Athens Greece: ACM, Aug. 2021, pp. 416–428, ISBN: 978-1-4503-8562-6.

- [16] V. Lenarduzzi and A. Panichella, “Serverless testing: Tool vendors’ and experts’ points of view,” *IEEE Software*, vol. 38, no. 1, pp. 54–60, Jan. 2021.
- [17] A. Mampage, S. Karunasekera, and R. Buyya, “Deadline-aware dynamic resource management in serverless computing environments,” in *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2021, pp. 483–492.
- [18] X. Liu, J. Wen, Z. Chen, *et al.*, “Faaslight : General application-level cold-start latency optimization for function-as-a-service in serverless computing,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 5, pp. 1–29, Sep. 2023.
- [19] K. Kritikos and P. Skrzypek, “A review of serverless frameworks,” in *2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion)*, Zurich: IEEE, Dec. 2018, pp. 161–168, ISBN: 978-1-72810-359-4.
- [20] J. M. Hellerstein, J. Faleiro, J. E. Gonzalez, *et al.*, “Serverless computing: One step forward, two steps back,” 2019.
- [21] S. Eismann, J. Scheuner, E. V. Eyk, *et al.*, “The state of serverless applications: Collection, characterization, and community consensus,” *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 4152–4166, Oct. 2022.
- [22] S. Eismann, J. Scheuner, E. Van Eyk, *et al.*, “Serverless applications: Why, when, and how?” *IEEE Software*, vol. 38, no. 1, pp. 32–39, Jan. 2021.
- [23] J. Spillner, *Quantitative analysis of cloud function evolution in the aws serverless application repository*, May 2019. arXiv: 1905.04800 [cs].
- [24] R. Krishnamurthi, A. Kumar, S. S. Gill, and R. Buyya, “Serverless computing: New trends and research directions,” in *Serverless Computing: Principles and Paradigms*, R. Krishnamurthi, A. Kumar, S. S. Gill, and R. Buyya, Eds., vol. 162, Cham: Springer International Publishing, 2023, pp. 1–13, ISBN: 978-3-031-26632-4 978-3-031-26633-1.
- [25] K. Ka, N. J. Yadwadkar, and C. Kozyrakis, “Hermod: Principled and practical scheduling for serverless functions,” 2022.
- [26] K. Kallas, H. Zhang, R. Alur, S. Angel, and V. Liu, “Executing microservice applications on serverless, correctly,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. POPL, pp. 367–395, Jan. 2023.
- [27] Y. K. Kim, M. R. HoseinyFarahabady, Y. C. Lee, and A. Y. Zomaya, “Automated fine-grained cpu cap control in serverless computing platform,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 10, pp. 2289–2301, 2020.
- [28] A. Suresh and A. Gandhi, “Fnsched: An efficient scheduler for serverless functions,” *Proceedings of the 5th International Workshop on Serverless Computing*, 2019.
- [29] A. Suresh, G. Somashekar, A. Varadarajan, V. R. Kakarla, H. Upadhyay, and A. Gandhi, “Ensure: Efficient scheduling and autonomous resource management in serverless environments,” in *2020 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, 2020, pp. 1–10.
- [30] K. Suo, J. Son, D. Cheng, W. Chen, and S. Baidya, “Tackling cold start of serverless applications by efficient and adaptive container runtime reusing,” in *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, 2021, pp. 433–443.
- [31] K. Solaiman and M. A. Adnan, “Wlec: A not so cold architecture to mitigate cold start problem in serverless computing,” in *2020 IEEE International Conference on Cloud Engineering (IC2E)*, 2020, pp. 144–153.
- [32] R. Cordingly, W. Shu, and W. J. Lloyd, “Predicting performance and cost of serverless computing functions with saaf,” in *2020 IEEE Intl Conf on Dependable, Autonomic*

- and Secure Computing, *Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*, 2020, pp. 640–649.
- [33] N. Akhtar, A. Raza, V. Ishakian, and I. Matta, “Cose: Configuring serverless functions using statistical learning,” in *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*, 2020, pp. 129–138.
- [34] S. Eismann, J. Grohmann, E. van Eyk, N. Herbst, and S. Kounev, “Predicting the costs of serverless workflows,” New York, NY, USA: Association for Computing Machinery, 2020, pp. 265–276, ISBN: 9781450369916.
- [35] C. Lin and H. Khazaei, “Modeling and optimization of performance and cost of serverless applications,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 3, pp. 615–632, 2021.
- [36] K. Mahajan, D. Figueiredo, V. Misra, and D. Rubenstein, “Optimal pricing for serverless computing,” in *2019 IEEE Global Communications Conference (GLOBECOM)*, 2019, pp. 1–6.
- [37] J. Spillner, “Resource management for cloud functions with memory tracing, profiling and autotuning,” *Proceedings of the 2020 Sixth International Workshop on Serverless Computing*, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:230507520>.
- [38] B. Carver, J. Zhang, A. Wang, A. Anwar, P. Wu, and Y. Cheng, “Wukong: A scalable and locality-enhanced framework for serverless parallel computing,” *Proceedings of the 11th ACM Symposium on Cloud Computing*, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:221856538>.
- [39] A. Christidis, S. Moschogiannis, C.-H. Hsu, and R. Davies, “Enabling serverless deployment of large-scale ai workloads,” *IEEE Access*, vol. 8, pp. 70 150–70 161, 2020.
- [40] V. Yussupov, U. Breitenbücher, A. Kaplan, and F. Leymann, “Seaport: Assessing the portability of serverless applications,” in *International Conference on Cloud Computing and Services Science*, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:219331253>.
- [41] A. Fuerst and P. Sharma, “Faascache: Keeping serverless computing alive with greedy-dual caching,” in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Virtual USA: ACM, Apr. 2021, pp. 386–400, ISBN: 978-1-4503-8317-2.